PLATEFORME FINANCE & TRÉSORERIE

Architecture technique détaillée

Version 2 · Next.js · FastAPI · PostgreSQL · Docker Compose

VERSION	DATE	PUBLIC
V2 (remplace la V1)	**21/10/2026**	**Développeurs, exploitation**

Ce document décrit l'application telle qu'elle est construite au 21/10/2026 : stack, organisation du code, services, API, base de données, sécurité, performances, tests et déploiement. Le fichier CLAUDE.md du dépôt reste la référence détaillée de chaque règle.

Sommaire

01 Vue d'ensemble et chiffres clés

02 Stack et bibliothèques

03 Déploiement Docker Compose

04 Frontend Next.js

05 Backend FastAPI

06 Services métier

07 API REST

08 Base de données PostgreSQL

09 Pipeline d'import

10 Moteur de rapprochement

11 Frais bancaires et export Sage

12 Sécurité

13 Audit et traçabilité

14 Qualité : tests et intégration continue

15 Exploitation et évolutions

1. Vue d'ensemble

Une application web en trois conteneurs. Le navigateur ne parle qu'à l'API ; l'API applique la sécurité et toutes les règles métier ; PostgreSQL stocke les données normalisées et l'historique.

Navigateur	Trésorerie, Comptabilité, Responsable, Direction, Administrateur

▼ HTTPS / REST (JSON, multipart pour les fichiers)

Frontend Next.js	App Router, React 19, TypeScript, Tailwind · port 3000

▼ apiFetch + jeton Bearer

Backend FastAPI	api → services → repositories → models · port 8000

▼ SQLAlchemy 2 / psycopg 3

PostgreSQL 17	34 tables, migrations Alembic, contraintes et déclencheurs · port 5434 (hôte)

34	35	~95	1 331
tables	migrations	points d'API	tests automatiques

Règle d'architecture. SIMTIS est une couche d'intégration, de contrôle et de pilotage, pas un second Sage. Le frontend ne contient aucune règle métier ni calcul financier ; il ne parle jamais à PostgreSQL.

2. Stack et bibliothèques

Couche	Technologie	Rôle
Frontend	Next.js 16 (App Router), React 19, TypeScript 5	Pages, routage, rendu.
	Tailwind CSS 4 + jetons `simtis-*`	Design system teal (`globals.css`).
	Recharts 3	Graphiques (évolution, donut, barres, courbes KPI).
	lucide-react	Icônes (pas d'emoji).
	clsx, tailwind-merge	Composition des classes.
Backend	Python 3.11, FastAPI, Uvicorn	API REST, validation, documentation OpenAPI.
	Pydantic 2, pydantic-settings	Schémas d'entrée / sortie (`extra="forbid"`), configuration.
	SQLAlchemy 2, Alembic	ORM et migrations.
	psycopg 3	Pilote PostgreSQL.
	PyJWT, pwdlib (argon2)	Jetons d'accès, hachage des mots de passe.
	openpyxl, xlrd	Lecture `.xlsx` et `.xls` ; écriture des exports Excel.
	python-multipart	Réception des fichiers.
	cydifflib	Ressemblance des libellés (difflib compilé, ~4× plus rapide, mêmes résultats).
Base	PostgreSQL 17	Données, contraintes, déclencheurs, audit non modifiable.
Qualité	pytest, httpx, ruff · Vitest, ESLint, Prettier, tsc	Tests et contrôles, lancés aussi en CI.

3. Déploiement Docker Compose

Service	Image	Port	Rôle
`db`	postgres:17-alpine	127.0.0.1:5434 → 5432	Base ; volume persistant.
`backend`	python:3.11-slim	8000	Au démarrage : migrations Alembic, seeds de référence, puis l'API. Contrôle de santé `GET /api/health`.
`frontend`	node:22-slim	3000	`next dev --webpack` avec scrutation des fichiers (Windows) ; `node_modules` dans un volume nommé.

```
docker compose up --build          # tout démarrer (sans .env)
docker compose up -d --wait        # en arrière-plan, attendre l'état sain
```

- **Accès réseau local** (désactivé par défaut) : `BIND_ADDRESS=0.0.0.0`, `CORS_ORIGIN_REGEX`, `ALLOWED_DEV_ORIGINS` dans `.env` ; règle de pare-feu Windows 3000/8000. Le navigateur appelle l'API sur l'hôte de la page, port 8000. La base reste sur 127.0.0.1.
- Redis ou un worker ne sont pas nécessaires : les plus gros imports (50 000 lignes) prennent 7 s à l'analyse et 19 s à la confirmation.

4. Frontend Next.js

4.1 Organisation

Dossier	Contenu
`app/(app)/`	Routes protégées : dashboard, banques, comptes, releves, position-bancaire, ecritures, rapprochement, rapports, historique, administration, ecarts (masqué), design-system ; `app/login`.
`components/`	Par module (dashboard, releves, rapprochement, position, accounts…) + `ui/` (Card, DataTable, Modal, Button, Field, StatusBadge, Pagination, états chargement / vide / erreur) + `layout/` (AppShell, barre latérale, en-tête, cloche).
`services/`	Une fonction par appel d'API, typée.
`lib/`	Client API (`api.ts`), règles d'affichage pures et testées (`format.ts` , `reconciliation.ts` , `statementLines.ts` , `frais.ts` …), permissions, drapeaux (`features.ts`).
`types/`	Types TypeScript des réponses de l'API.

4.2 Règles

- Tout appel passe par `apiFetch` : jeton Bearer, un rafraîchissement sur 401 puis retour à `/login`, message « Requête refusée : données invalides. » pour un 422 sans message métier ; `apiDownload` pour les fichiers.
- Le jeton d'accès reste en mémoire, jamais dans `localStorage`.
- Argent : chaînes exactes (`"500000.00"`) ; les sommes d'écran se font en centimes `BigInt`, jamais en `float`. `formatAmount()` pour tout montant affiché, `formatCompact()` (« 18 M ») pour les axes.
- Société active : `useCompany()` ; chaque écran ne lit qu'une société.
- Menu filtré par `ROUTE_PERMISSIONS` (`lib/permissions.ts` , copie vérifiée par un test de `core/permissions.py`). Masquer un menu ne protège rien : l'API vérifie.
- Les tableaux Banques, Devises et Prévisions ont une structure imposée et n'utilisent pas `DataTable`.

5. Backend FastAPI



- Les erreurs métier sont levées par les services (`NotFoundError` → 404, `ConflictError` → 409, message en français) et converties par `main.py` en `{"detail": message}`.
- Un nouveau routeur s'ajoute au `protected_router` de `api/router.py` (401 sans session) ; un nouveau modèle s'importe dans `models/__init__.py` (sinon Alembic ne le voit pas).
- Argent : `NUMERIC(18,2)` en base, `Decimal` en Python. Formules de position en fonctions pures (`position_service`).
- Date métier : `business_today()`, fuseau Africa/Casablanca.
- Commandes : `python -m app.seeds [--demo]`, `python -m app.cli create-user`, `python -m app.cli recalculer-soldes`.

6. Services métier

Service	Responsabilité
`auth_service`	Connexion, sessions, rafraîchissement, déconnexion, verrouillage.
`bank_service`, `account_service`	Banques, comptes, journaux Sage, suppression d'un compte sans historique.
`balance_service`	Soldes saisis, chiffres d'un compte (crédit et position disponibles).
`import_file`	Lecture commune des fichiers : feuille, en-tête, colonnes, synonymes, abréviations, limites.
`normalization_service`	Fonctions pures : dates, montants (« 213,68* »), libellés, références, empreintes, pointage, opérations en cours.
`import_service`	Import des relevés : analyse, soldes, sens, doublons, confirmation, export standard, suppression des opérations en cours.
`accounting_import_service`	Import Sage : lignes banque des journaux de banque, montant devise, doublons, retours d'écritures SIMTIS.
`position_service`, `position_banques_service`	Formules en `Decimal`, tableau Banques, soldes par compte (relevé, saisie, opérations en cours).
`saisie_service`	Saisies Devises et Prévisions.
`reconciliation_scoring`	Règles pures du moteur : types, dates, score, filtrage rapide des paires, attribution, recherche élargie.
`reconciliation_groupes`	Groupes N → 1 (principal + commission + TVA), score des rapprochements manuels.
`reconciliation_service`	Lancement, propositions, ambiguës, validation, rejet, annulation, manuel, codes, historique, lettrage.
`frais_service`	Frais bancaires : familles, ventilation TVA, comptes généraux, N° pièce, validation, écritures manuelles, export CSV, justificatif.
`discrepancy_service`	Écarts (module masqué côté écran).
`dashboard_service`	KPI, évolution, frais, comptes, tâches, alertes, filtre banque.
`admin_service`, `parametres_service`	Utilisateurs, rôles, gardes ; paramètres du rapprochement, modèles de colonnes.
`historique_service`, `audit_service`	Lecture du journal ; écriture des entrées d'audit (masquage des secrets).
`rapports_service`	Exports Excel position et rapprochement.

7. API REST

Préfixe `/api`. Toutes les routes sauf la santé et l'authentification exigent une session ; chaque route métier exige une permission.

Module	Points d'entrée principaux
Auth	`POST /auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`
Référentiel	`GET /companies`, `/currencies`, `/pointage-types`, `/health`
Banques, comptes	`/banks` (CRUD, statut), `/accounts` (CRUD, statut, `DELETE`), `/accounts/{id}/balances`, `PUT …/balances/{jour}`
Relevés	`POST /statements/import/analyse`, `/import/confirm` ; `GET /statements`, `/statements/{id}/transactions`, `/statements/accounts/{id}` et leurs exports ; `PATCH /statements/transactions/{id}` ; `POST /statements/transactions/supprimer-en-cours`
Sage	`POST /accounting/import/analyse`, `/import/confirm` ; `GET /accounting/entries`, `/entries/{id}`, `/imports`
Position	`GET /position/banques`, `/position/devises/soldes` ; `GET` / `PUT /position/devises`, `/position/previsions`
Rapprochement	`POST /reconciliation/run`, `/run-elargie` ; `GET /transactions`, `/proposals`, `/ambiguous`, `/ambiguous-entries`, `/candidates`, `/history`, `/export` ; `POST /matches` (manuel), `/matches/validate-batch`, `/matches/reject-batch`, `/matches/{id}/validate`, `/reject`, `DELETE /matches/{id}` ; « aucune correspondance » unitaire et en lot
Frais	`GET /frais`, `POST /frais/valider`, `/manuelle`, `/ecarter`, `/reproposer`, `DELETE /frais/ecritures/{id}`, `GET /frais/export` (CSV), `/frais/justificatif`
Dashboard	`GET /dashboard`, `/dashboard/alertes`
Écarts	`/discrepancies` (liste, création, génération, modification, clôture)
Administration	`/admin/users`, `/admin/roles`, `/admin/permissions` ; `/parametres/rapprochement`, `/parametres/mappings`
Historique, rapports	`GET /audit`, `/audit/filtres`, `/audit/{id}` ; `GET /rapports/position`, `/rapports/rapprochement`

Les imports sont sans état : la confirmation renvoie le fichier, qui est ré-analysé et re-contrôlé par le serveur. Les lignes choisies voyagent en partie JSON (jusqu'à 60 Mo), car un champ texte multipart est limité à 1 Mo.

8. Base de données PostgreSQL

8.1 Tables (34)

Domaine	Tables
Référentiel	`companies`, `banks`, `currencies`, `exchange_rates`, `pointage_types`, `forecast_categories`
Comptes et soldes	`bank_accounts`, `bank_account_balances`, `balance_checks`
Imports	`import_batches`, `column_mappings`, `bank_statements`, `bank_transactions`, `accounting_entries`
Rapprochement	`reconciliation_matches`, `reconciliation_match_items`, `reconciliation_rules`, `reconciliation_code_counters`
Frais	`frais_piece_counters`, `frais_lignes`, `frais_jours_ecartes`
Écarts, prévisions	`discrepancies`, `cash_forecasts`, `saisies_devises`, `saisies_previsions`, `saisies_previsions_jour`
Sécurité, audit	`users`, `roles`, `permissions`, `user_roles`, `role_permissions`, `user_sessions`, `audit_logs`

8.2 Relations

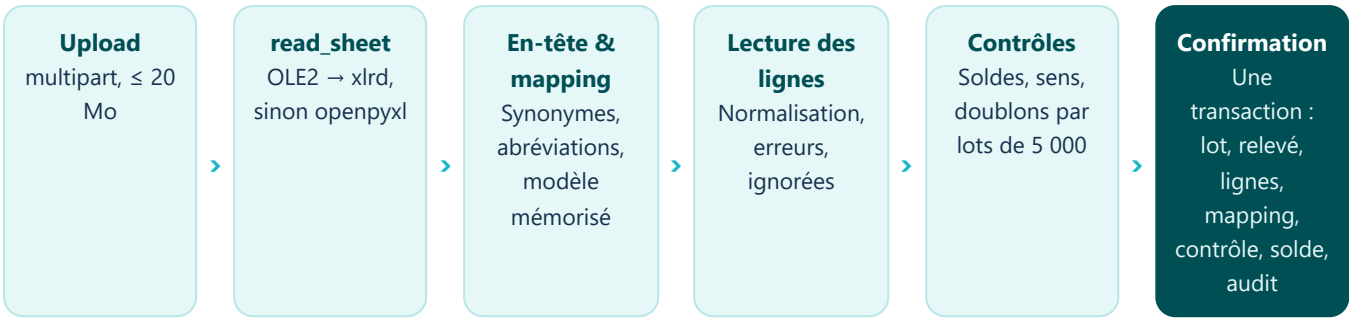
```
Company 1—N BankAccount ;  Bank 1—N BankAccount 1—N BankStatement 1—N BankTransaction
BankTransaction N—N AccountingEntry  via  reconciliation_matches + reconciliation_match_items
(1-1, 1-N, N-1, N-N)
AccountingEntry (origine Sage ou SIMTIS) 1—N frais_lignes
```

8.3 Garanties en base

- Un compte actif par (société, banque, devise, type) : index unique partiel ; un journal Sage par compte actif et par société : déclencheur.
- Une seule correspondance active par opération et par écriture : index uniques partiels sur `reconciliation_match_items.actif`.
- Code de rapprochement unique par société ; compteurs sans trou (`INSERT … ON CONFLICT … RETURNING`).
- CHECK : décision tracée, montant devise de même signe, famille de frais posée exactement pour les écritures SIMTIS, TVA et taux cohérents, opération remplacée seulement si en cours…
- `audit_logs` non modifiable (déclencheur) : ni mise à jour ni suppression.

Migrations. 35 migrations Alembic appliquées à chaque démarrage. Une migration générée se relit toujours (elle ignore déclencheurs et fonctions). `alembic check` ne compare pas les CHECK : chaque contrainte a son test dans `tests/test_constraints.py`. Les seeds créent sans jamais modifier ni supprimer.

9. Pipeline d'import



- Empreinte de ligne (hash) pour les doublons ; rang dans le fichier pour distinguer deux lignes identiques ; empreinte définitive gardée pour une opération en cours complétée.
- Ordre chronologique : date, relevé, puis `bank_transactions.ordre` (rang dans le fichier), utilisé pour le solde du jour et les soldes calculés.
- Soldes calculés : `solde_calcule = true` ; recalcul possible par `app.cli recalculer-soldes`.
- Mesures : 50 000 lignes analysées en 7 s, confirmées en 19 s ; export Sage de même taille 3 s / 10 s.

10. Moteur de rapprochement

10.1 Étapes d'un lancement

1. Verrou de la société en `FOR UPDATE NOWAIT` (un second lancement reçoit 409 immédiatement).
2. Lecture des opérations libres (hors frais et opérations en cours) et des écritures libres de la période (≤ 5 ans).
3. Paires comparables : même compte, sens opposé, types compatibles, fenêtre de dates (± 10 j, 180 j à montant et numéro égaux).
4. Score par `reconciliation_scoring`, avec rejet rapide prouvé sous le seuil (points sans libellé, référence, bornes supérieures du ratio de libellé).
5. Attribution du meilleur score au plus faible ; égalité → ambiguë ; second score gardé pour « Concurrente proche ».
6. Groupes N → 1 (principal + commission + TVA 10 %) ; remplacement des seules propositions en attente du moteur ; audit.

10.2 Performances mesurées (données de développement)

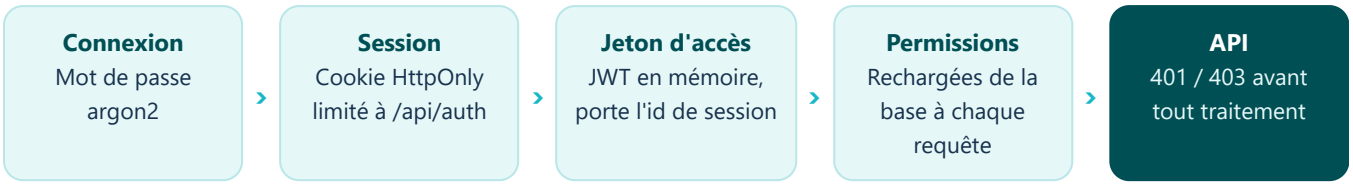
Traitement	Avant	Après
Lancement complet, une année, tous comptes	dizaines de min	99 s
Étape 1 → 1	290 s	89 s
Recherche élargie (1 an)	—	1,2 s
Relance ciblée depuis une carte ambiguë	12,5–14 s	0,35–1,9 s
Carte « Frais bancaires », une année	86 s	1,9 s
Validation en lot de 1 513 propositions	—	≈ 6 s

La grille et les seuils sont dans `reconciliation_rules`, modifiables par l'écran Paramètres ; ils s'appliquent au lancement suivant.

11. Frais bancaires et export Sage

- Propositions **calculées, jamais stockées** (`GET /frais`) ; la validation recalcule tout côté serveur, tout ou rien.
- Écriture : `accounting_entries` avec `origine = 'SIMTIS'`, `famille_frais`, N° pièce de `frais_piece_counters` (par société, banque, mois), plus un rapprochement validé d'origine « Frais ».
- Lignes saisies par l'utilisateur dans `frais_lignes` ; journées écartées dans `frais_jours_ecartes`.
- Export CSV : séparateur « ; », Windows-1252, virgule décimale, dates JJ/MM/AAAA, 15 colonnes de l'import paramétrable Sage ; marque `exportee_le`. Refusé pour un compte EUR / USD (pas de taux).
- Seuil `SEUIL_MANUEL` = 200 DH pour les familles COM et FRAIS ; justificatif Excel pour la banque.

12. Sécurité



- Les permissions ne sont jamais dans le JWT : un changement de rôle s'applique tout de suite ; déconnexion, changement de mot de passe et désactivation coupent l'accès immédiatement.
- Tolérance d'horloge de 60 s au décodage (l'horloge de la VM Docker recule parfois).
- `tests/test_permissions.py` échoue si une route répond sans jeton hors liste publique.
- Codes de référence (banques, sociétés, rôles) immuables ; schémas d'entrée stricts (champ inconnu → 422).
- CORS limité aux origines configurées.

13. Audit et traçabilité

- `audit_service.log(db, user_id, action, entite, entite_id, avant, apres, ip)` dans la même transaction que la modification, puis un seul commit.
- Masquage automatique des mots de passe et jetons ; `Decimal` stockés en texte exact.
- Plus de 60 actions tracées : imports, corrections, suppressions d'opérations en cours, propositions, validations, rejets, annulations, rapprochements manuels et élargis, comptabilisations, exports, administration, paramètres.
- Lecture par `/api/audit` (50 par page, < 0,05 s sur 36 000 entrées) ; le détail n'est chargé qu'à l'ouverture d'une ligne.

14. Qualité : tests et intégration continue

Côté	Outils	Tests	Principe
Backend	pytest, httpx, ruff	1 070	Base dédiée `simtis_test`, créée et migrée par `conftest.py` (la vraie migration est testée), chaque test dans une transaction annulée ; `assert_rejected` pour les contraintes.
Frontend	Vitest, tsc, ESLint, Prettier, build	261	Règles pures de `lib/` (montants en BigInt, filtres, sélection, format).

```
docker compose run --rm backend ruff check . && ruff format --check . && pytest
npm run lint && npm run format:check && npm run typecheck && npm test && npm run build
```

La CI (`.github/workflows/ci.yml`) lance les mêmes contrôles ; une tâche n'est terminée que si tout passe.

15. Exploitation et évolutions

15.1 Mise en production (à faire)

- Images de production : `next build` + `next start`, Uvicorn sans rechargement, variables d'environnement de production (`APP_ENV=production` refuse les données de démo).
- HTTPS par un reverse proxy (Nginx ou Caddy) sur le serveur local ; base non exposée.
- Sauvegarde quotidienne de PostgreSQL (`pg_dump`) et test de restauration.
- Comptes réels créés par `python -m app.cli create-user`.

15.2 Évolutions prévues

Évolution	Remarque
P14 Prévisions	Forecast service, position prévisionnelle en cascade, alertes ; bloqué par une question métier.
P9 Devises calculées	Taux de change saisis avec date, équivalent MAD, export des frais des comptes en devise.
P13.3 1 → N automatique	2 à 5 écritures pour une opération, somme exacte.
Recherche globale	Champ de l'en-tête masqué tant qu'il n'est pas construit.
Réactivation des Écarts	Drapeau `ECARTS_ACTIFS` dans `lib/features.ts`.
Worker asynchrone	Seulement si les volumes l'exigent.